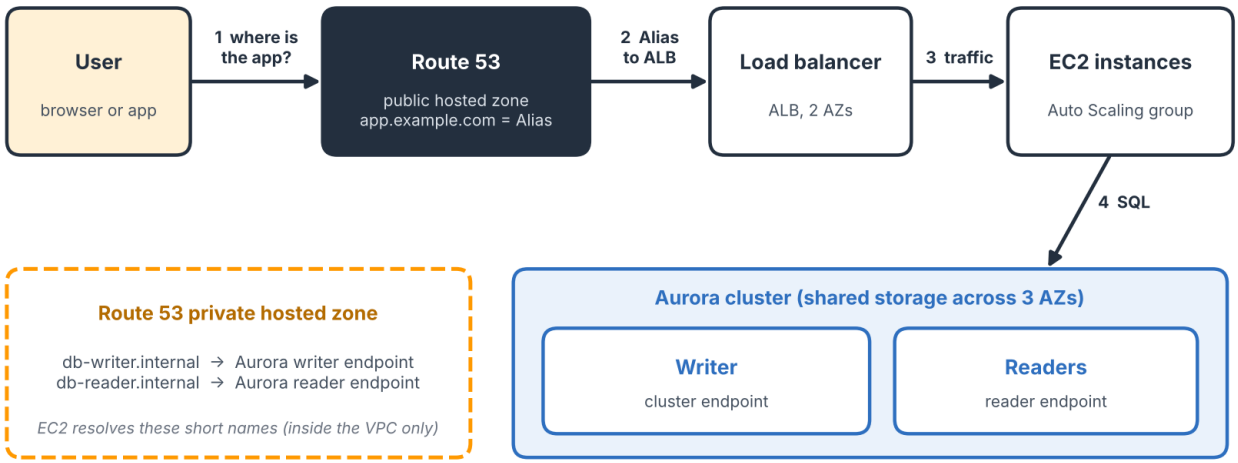
DNS, Amazon Route 53 and Amazon Aurora

Study notes with real-world examples and hands-on practice

How this fits together

DNS, Route 53 and Aurora look like three separate topics, but in a real application they work as one chain. Users type a **name**; **DNS** turns it into an address; **Route 53** is where AWS keeps those name records; and **Aurora** is the database the application behind that name talks to, again *by name* (a DNS name for the database endpoint). The diagram shows the whole picture; the sections below explain each box.



Note: DNS only tells the browser *where* the load balancer is (steps 1 and 2). The actual web traffic then goes straight from the browser to the load balancer (step 3).

PART 1: DNS FUNDAMENTALS

1.1 What DNS does

Computers find each other by IP address (like `203.0.113.10`), but people remember names (like `example.com`). The **Domain Name System** is the global phone book that turns names into IP addresses.

1.2 Parts of a domain name

```
www . example . com .
 |      |       |    |
 |      |       |    +-- root (the invisible final dot)
 |      |       +------- TLD: top-level domain (.com, .in, .org)
 |      +--------------- SLD: the name you register (example)
 +---------------------- subdomain (www, app, api, mail)
```

The full name, such as `app.example.com`, is called a **FQDN** (fully qualified domain name).

1.3 Who is who

Term	Meaning
Domain registrar	The company where you **buy** the domain (Route 53, GoDaddy, Namecheap and others). The registrar records which name servers are responsible for your domain.
DNS service / hosted zone	The place that **stores your records** and answers questions about them. Route 53 does this. You can buy a domain at one company and host its DNS at another.
Name server (NS)	A server that holds DNS records. Each domain lists several, for reliability.
Authoritative server	The name server that holds the **official** answer for your domain.
Recursive resolver	The server (usually your ISP's, or 8.8.8.8, 1.1.1.1) that does the lookup **for you** and caches the answer.
Zone	The set of records for one domain (or part of it).

1.4 What happens when you open a website

1. Your browser checks its own cache, then the operating system's cache. If the answer is there, it stops.
2. It asks the **recursive resolver** (your ISP or a public one).
3. If the resolver has no cached answer, it asks a **root server**: "who handles .com?"
4. The root points to the **.com TLD servers**, which say: "the name servers for example.com are ns-123.awsdns-45.com ..."
5. The resolver asks that **authoritative name server** (Route 53) for `www.example.com`, and gets back an IP address.
6. The resolver returns the answer to your browser and **caches** it for the record's **TTL**. The browser connects to the IP.

TTL (time to live) is how many seconds a resolver may keep an answer before asking again. A short TTL (60 seconds) means changes spread fast but you get more DNS queries. A long TTL (86400 = 1 day) means fewer queries but slow changes.

1.5 Record types you must know

Type	What it stores	Example
A	Name → IPv4 address	`app.example.com → 203.0.113.10`
AAAA	Name → IPv6 address	`app.example.com → 2001:db8::1`
CNAME	Name → another **name** (an alias to a name)	`www.example.com → example.com`
MX	Where email for the domain goes (with a priority)	`10 mail.example.com`
TXT	Free text, used for verification and email security (SPF, DKIM, DMARC)	`"v=spf1 include:... ~all"`
NS	The name servers responsible for the domain	`ns-123.awsdns-45.com`
SOA	Start of authority: zone admin information, created automatically	(one per zone)

Key rule: a **CNAME cannot be used at the zone apex** (the bare domain, like `example.com`), because the apex must also hold NS and SOA records. This is exactly the problem Route 53's **Alias** record solves.

Real-world example: moving a website with no downtime
A company is moving its site to a new server. **Two days before** the move, they lower the A record's TTL from 86400 to 60 seconds. On move day they change the IP; within about a minute almost every user sees the new server. Afterwards they raise the TTL again. If they had changed the IP without lowering the TTL first, some users could keep reaching the old server for up to a day.

Real-world example: setting up company email
To use a hosted email service, you add **MX** records that point to the provider, and **TXT** records (SPF, DKIM, DMARC) so receiving servers trust your messages. Missing or wrong TXT records are the usual reason company email lands in spam.

1.6 Practice: be a DNS detective (free, on your Windows laptop)

Open **Command Prompt** and try these. You are asking real DNS servers real questions, at no cost. Replace the domain with any site you like.

```
nslookup example.com
nslookup -type=NS example.com
nslookup -type=MX gmail.com
nslookup -type=TXT example.com
nslookup www.example.com 8.8.8.8      (ask Google's resolver instead of your ISP's)
ipconfig /displaydns                  (see what your own computer has cached)
ipconfig /flushdns                    (clear your local DNS cache)
```

- Look at the **name servers** of a few big sites. Can you tell which ones use Route 53? (Route 53 names look like `ns-123.awsdns-45.com`.)
- Run `nslookup` twice on the same name. Does the address change? Large sites often return different addresses for load distribution.
- Check the MX records of two different email providers and compare them.

PART 2: AMAZON ROUTE 53

2.1 What it is

Route 53 is AWS's **highly available DNS service** and also a **domain registrar**. The name comes from DNS using port **53**. It is the only AWS service with a **100% availability SLA**. It can: register domains, hold your DNS records, check whether your servers are healthy, and route users to different servers using several policies.

2.2 Hosted zones

Type	Used for	Visible to
Public hosted zone	Records for the internet (your website, API, email)	Anyone on the internet
Private hosted zone	Names that work only inside one or more of your VPCs (for example `db-writer.internal`)	Only the VPCs you attach

- When you create a public hosted zone, AWS gives you **4 name servers (NS)**. If your domain is registered *elsewhere*, copy those 4 values into the registrar's name-server settings. Changes can take minutes to a couple of days to spread.
- Private hosted zones require the VPC options **DNS resolution** and **DNS hostnames** to be turned on.
- **Cost:** a hosted zone costs about $0.50 per month (a zone deleted within 12 hours of creation is not charged), plus a small charge per million queries. Domain registration is a yearly fee that depends on the domain ending. Check the Route 53 pricing page for current numbers.

2.3 Alias records vs CNAME

	CNAME	Alias (Route 53 only)
Works at the apex (example.com)?	**No**	**Yes**
Can point to	Any other DNS name	Certain **AWS resources**: ALB/NLB, CloudFront, S3 website endpoint, API Gateway, Elastic Beanstalk, Global Accelerator, VPC interface endpoints, or another record in the same hosted zone
Query cost	Charged	**Free** for queries to AWS resources
Health integration	Separate health check	Can use **Evaluate Target Health**
TTL	You set it	Not set by you; Route 53 manages it
Record type shown as	CNAME	A (or AAAA)

Rule of thumb: if the target is an AWS resource, use an **Alias**. Otherwise use a CNAME (and never at the apex). You **cannot** use an Alias to point at the DNS name of an **EC2 instance**.

2.4 Routing policies

A routing policy decides *which answer* Route 53 gives when several records share a name.

Policy	How it decides	Typical use
Simple	One record. If it has several values, the client picks one	A single website
Weighted	Splits traffic by weight (0 to 255). Weight 0 stops sending traffic	Canary releases, A/B tests, gradual migration
Latency	Sends the user to the AWS Region with the lowest network latency	Global apps with servers in several Regions
Failover	Primary record while healthy; otherwise secondary. Needs a health check on the primary	Active-passive disaster recovery
Geolocation	By the user's location (country or continent), with a default record	Legal or language rules, content restrictions
Geoproximity	By distance to resources, with a bias to shift traffic toward or away from a site	Shifting load between sites
IP-based	By the client's IP range (CIDR) that you define	Send a specific ISP or office to a chosen endpoint
Multi-value answer	Returns up to 8 healthy records at random	Simple client-side load spreading with health checks

Latency is not geography. Latency routing uses measured network speed to AWS Regions. Geolocation uses where the user *is*. They can give different answers.

2.5 Health checks

- Route 53 can check an endpoint (HTTP, HTTPS or TCP) from several locations around the world, about every 30 seconds (or every 10 seconds for a higher price).
- **Three kinds:** an endpoint check; a **calculated** check that combines several others; and a check that watches a **CloudWatch alarm**. Use the alarm type for **private** resources, since Route 53's checkers are on the public internet and cannot reach a private IP.
- An unhealthy record is removed from answers (for failover, weighted, latency and multi-value policies).

Real-world example: gradual release with weighted routing
A shop deploys version 2 of its app to a new target. It creates two Alias records for `shop.example.com`: version 1 with weight **95** and version 2 with weight **5**. If dashboards look fine, it moves to 50/50, then 0/100. If errors appear, it sets version 2's weight to **0**, and traffic returns to version 1 in about a minute.

Real-world example: maintenance page with failover
Primary: Alias to the production load balancer, with a health check. Secondary: Alias to an **S3 static website** with a friendly "we'll be right back" page (the project you already built). If the load balancer fails its health check, users automatically see the S3 page instead of an error.

Real-world example: users in India and Europe
An app runs in Mumbai (ap-south-1) and Stockholm (eu-north-1). With **latency routing**, users in India usually reach Mumbai and users in Europe reach Stockholm, without the application doing anything.

Real-world example: private names for internal systems
Instead of putting the long database endpoint into every application's config, create a **private hosted zone** `internal.mycompany.com` with a record `db-writer` pointing to the endpoint. When the database changes (a restore, a migration), you update **one DNS record**, not every application.

2.6 Hands-on lab: a private hosted zone (no domain needed)

Goal: make an EC2 instance reach another instance by a friendly name inside your VPC. **Cost:** a hosted zone deleted within 12 hours is not charged; instances stay in the Free Tier. Delete everything afterwards.

1. Launch **one** t3.micro EC2 instance (Amazon Linux 2023) with the default settings. You will use **EC2 Instance Connect** to log in, so keep SSH (port 22) allowed. Copy its **private IPv4 address** (starts with `172.31.`).
2. Open **Route 53** → **Hosted zones** → **Create hosted zone**. Domain name: `lab.internal`. Type: **Private hosted zone**. Choose Region **eu-north-1** and your **default VPC**. Click Create.
3. Click **Create record**. Record name: `web`. Type: **A**. Value: the instance's private IP. TTL: **60**. Create.
4. Connect to the instance with EC2 Instance Connect and run:

```
nslookup web.lab.internal
ping -c 2 web.lab.internal
```

5. You should see the private IP you entered. **Now change the record's value** to a different private IP, wait 60 seconds, and run `nslookup` again to see the TTL in action.
6. If the name does not resolve, check that the VPC has **DNS resolution** and **DNS hostnames** enabled, and that the hosted zone is attached to the right VPC.
7. **Clean up:** delete the record, delete the hosted zone, and terminate the instance.

If you own a domain: create a **public** hosted zone for it, copy the 4 NS values to your registrar, add an A record, and test with `nslookup yourdomain.com`. Delete the zone afterwards unless you want to keep it (about $0.50 per month).

PART 3: AMAZON AURORA

3.1 What it is

Aurora is a **cloud-built relational database** from AWS that is compatible with **MySQL** and **PostgreSQL**: applications and tools written for those databases work with little or no change. It is part of the RDS family but has a different storage design, which gives it better availability and faster failover than a standard RDS database.

3.2 How Aurora is built

- **Shared storage volume:** data is stored in **6 copies across 3 Availability Zones**. Writes need 4 of 6 copies and reads need 3 of 6, so Aurora tolerates losing a whole AZ.
- **Self-healing, auto-growing storage:** grows in 10 GB steps up to 128 TiB. You do not provision disk size.
- **One writer + up to 15 read replicas**, all attached to the *same* storage, so replicas lag very little (typically milliseconds).
- **Failover:** if the writer fails, Aurora promotes a replica, usually in well under a minute. Replicas have **failover priority tiers** (0 to 15).
- **Compute and storage are separate.** Adding a replica does not copy the data.

3.3 Endpoints (important for applications)

Endpoint	Points to	Use it for
Cluster (writer) endpoint	Always the current writer	All writes, and reads that must be up to date. **After a failover it automatically points to the new writer.**
Reader endpoint	Load-balanced across the read replicas	Read-only queries (reports, product browsing)
Custom endpoint	A group of instances you choose	E.g. a few large replicas for analytics only
Instance endpoint	One specific instance	Debugging only. **Do not use it in applications**, since the instance can change role at failover.

Two practical gotchas: (1) The reader endpoint balances **per connection**, not per query, so one long-lived connection stays on one replica. (2) Endpoints are **DNS names** with a very short TTL. After a failover the writer name points to a new IP, so an application that caches DNS for a long time (some Java setups do by default) keeps talking to the old, failed server. Configure a short DNS cache TTL in the application or driver.

3.4 Aurora vs standard RDS

	Standard RDS (MySQL / PostgreSQL)	Aurora
Storage	EBS volume you size; one copy per instance (+1 for Multi-AZ)	Shared, auto-growing volume, 6 copies in 3 AZs
Read replicas	Fewer; each keeps its own copy of the data	Up to 15; share the storage
Failover	Multi-AZ standby (not readable); usually 1 to 2 minutes	Replicas are readable and are the failover targets; usually under a minute
Cross-Region	Cross-Region read replicas	**Global Database**: replication lag typically under 1 second
Price	Lower per instance hour	Higher per instance hour; I/O options differ. Check pricing
Free Tier	Micro instances may be Free Tier eligible	**Generally not Free Tier**: avoid leaving clusters running

3.5 Key features

Feature	What it does
Auto Scaling for replicas	Adds or removes read replicas based on CPU or connections.
Serverless v2	Capacity scales automatically in small steps (measured in ACUs) with the load. Good for variable or unpredictable workloads and dev/test. Check current docs for minimum capacity and pause options.
Global Database	One primary Region plus read-only secondary Regions; typically under 1 second replication delay. A secondary can be promoted for disaster recovery in about a minute.
Cloning	Creates a copy of a large database in minutes, because it shares storage with the source until data changes (copy-on-write).
Backtrack (Aurora MySQL only)	Rewind the database to an earlier point in time *without* restoring from backup.
Backups	Continuous and automatic, retention 1 to 35 days, restore to any second in that window. A restore creates a **new cluster with a new endpoint**.
Security	Encryption at rest (KMS) must be chosen **when you create** the cluster; TLS in transit; keep it in **private subnets** with a security group that allows port 3306 (MySQL) or 5432 (PostgreSQL) only from the application's security group.

Real-world example: an online shop
The checkout service writes orders to the **cluster endpoint**. The product catalogue and search pages read from the **reader endpoint**, so heavy browsing never slows checkout. At sale time, Aurora Auto Scaling adds read replicas; afterwards they are removed to save cost.

Real-world example: monthly reports without hurting customers
The finance team's heavy report queries go to a **custom endpoint** that includes only two large replicas. Customer traffic keeps using the normal reader endpoint and is not affected.

Real-world example: safe testing of a risky change
Before a schema migration, a team creates a **clone** of the production database in minutes, runs the migration on the clone, and checks the result. Only then do they run it on production.

Real-world example: a restore that breaks the application
A team restores to a point before a bad deploy. The restore creates a **new cluster with a new endpoint**. Because every application uses the DNS name `db-writer.internal` from a Route 53 private zone, they just **update that one record** and everything reconnects.

Real-world example: disaster recovery for a regional outage
Primary cluster in Mumbai (ap-south-1) with a **Global Database** secondary in another Region. If Mumbai is unavailable, the team promotes the secondary and uses Route 53 **failover routing** (or a record update) to send the application to the new Region.

3.6 Practice: see writer vs reader (optional, costs money)

Warning: Aurora is generally **not** in the Free Tier, so a practice cluster costs a few cents per hour or more. If you build one, **delete it as soon as you finish**. For free practice, use a standard RDS **micro** instance and read the Aurora parts as theory. This lab is a guide I have written; I have not run it against a live AWS account.

1. **RDS → Create database → Standard create → Aurora (MySQL-compatible).** Choose a Dev/Test template and the smallest instance class offered. Add one Aurora Replica.
2. Put the cluster in your default VPC with a security group that allows port 3306 **only from your EC2 instance's security group**. Do not make it publicly accessible.

3 From an EC2 instance, install a MySQL client and connect to the **cluster endpoint** (copy it from the console):

```
sudo dnf install -y mariadb105
mysql -h <cluster-endpoint> -u admin -p

SHOW VARIABLES LIKE 'innodb_read_only';   -- OFF on the writer
SELECT @@aurora_server_id;                -- which instance you are on
CREATE DATABASE shop; USE shop;
CREATE TABLE orders (id INT PRIMARY KEY AUTO_INCREMENT, item VARCHAR(50));
INSERT INTO orders (item) VALUES ('book'),('pen');
```

4 Now connect to the **reader endpoint**. Run `SELECT * FROM shop.orders;` (works) and then try an `INSERT` (it should be **refused**, because readers are read-only). `innodb_read_only` should show ON.

5 In the console choose **Actions** → **Failover** and watch the writer and reader swap roles. Run `SELECT @@aurora_server_id;` on the **cluster endpoint** again. It now reports the other instance, yet your connection string did not change.

6 **Clean up:** delete the replica, then the cluster (skip the final snapshot for practice), and check that no snapshots or instances remain.

PART 4: PUTTING IT TOGETHER

Problem	Use
Point `example.com` (bare domain) at a load balancer	Route 53 **Alias** A record
Serve a "maintenance" page when the app is down	Route 53 **failover** routing + health check + S3 static site
Send users to the nearest Region	Route 53 **latency** routing
Roll out a new version to 5% of users	Route 53 **weighted** routing (or ALB weighted target groups)
Give internal systems friendly names	Route 53 **private hosted zone**
Scale reads without slowing writes	Aurora **reader endpoint** and replicas
Survive an AZ failure	Aurora (6 copies, 3 AZs, automatic failover)
Survive a Region failure	Aurora **Global Database** + Route 53 failover
Clone production for testing	Aurora **cloning**

Common mistakes

- Using a **CNAME at the apex**. Use an Alias.
- Hard-coding an Aurora **instance endpoint** in an application. Use the cluster or reader endpoint.
- Forgetting DNS **caching**: after a change, old answers can last until the TTL expires.
- Creating an Alias to an **EC2 DNS name** (not allowed) or expecting a public health check to reach a **private** resource.
- Leaving a hosted zone or an Aurora cluster running after practice, and getting a surprise bill.

Interview-style questions

Question	Short answer
Alias vs CNAME?	Alias works at the apex, is free for AWS targets, and integrates with health checks; CNAME cannot be used at the apex.
Latency vs geolocation routing?	Latency uses measured network speed to Regions; geolocation uses the user's country or continent.
What is TTL and why lower it before a migration?	How long answers are cached. A low TTL makes the cutover fast.
How does Aurora survive an AZ failure?	Six copies of data across three AZs plus replicas that are promoted automatically.
Which Aurora endpoint should an application use for writes?	The cluster (writer) endpoint, which always points to the current writer.
Why can a long DNS cache hurt a failover?	The app may keep connecting to the old writer's IP after the endpoint's DNS record changes.

Quick quiz

1. Why can't you use a CNAME for `example.com`? What do you use instead?
2. You want 10% of users to try a new version first. Which routing policy?
3. Which Route 53 hosted zone type is invisible to the internet?
4. How many read replicas can an Aurora cluster have, and how many copies of the data are stored?
5. Your application connects to an Aurora instance endpoint. What goes wrong at failover?
6. Which health check type would you use for a resource in a private subnet?
7. You restored an Aurora cluster to an earlier point. What do you have to change in the application, and how can DNS make that easier?
8. True or false: an Aurora reader endpoint load-balances every individual query.

Answers

1) The apex must also hold NS and SOA records, which a CNAME cannot coexist with; use an **Alias**. 2) **Weighted** (10 and 90). 3) **Private** hosted zone. 4) Up to **15** replicas; **6** copies across **3** AZs. 5) The instance may no longer be the writer, so writes fail; use the cluster endpoint. 6) A health check based on a **CloudWatch alarm**. 7) A restore creates a new cluster with a new endpoint; if apps use a private-zone name such as db-writer.internal, update that single DNS record. 8) **False**: balancing is per connection, not per query.